

The Schedule Is a Value: Comptime-Specialized Autotuning Without Forks

Hanzo AI — ML Systems

hanzo-kernel Paper 07.2 — part of the *One Source, Every Backend* series

Abstract

A single-source kernel DSL promises to kill the “one operation, N backends, N numeric behaviors” bug class by writing each op once. But there is a subtler place the same bug hides: the *schedule* — the tile size, vector width, unroll factor, and thread-block dimension a kernel is tuned with, which differs per device. The usual fix is to fork a hand-tuned kernel per device, and a fork is exactly what reintroduces the bug: two schedules, drifting independently, one of them one day reordering an accumulation and changing the numbers. This paper shows how to autotune without forking. The schedule is exposed as a set of compile-time (`#[comptime]`) knobs on the *one* source; each knob value is a distinct compiled kernel the lowering engine monomorphizes; a small set of these variants is timed once per (`device`, `op`, `shape`) and the winner cached to disk. Because the knobs never touch the arithmetic, *every variant is byte-identical* — the tuner buys speed and can never spend correctness. We give the three-value algebra (`Variant`, `Tuned`, `Tuner`), the per-device on-disk cache, the two shipped variant tables, and the offline tests that prove select/cache/reload works without a GPU.

1 The pain: a fork that drifts

Suppose you have written a matvec kernel once, in a portable DSL, and it lowers correctly to every backend. You are not done, because “correct everywhere” is not “fast everywhere.” The fastest schedule for an AMD iGPU is not the fastest for an NVIDIA datacenter part: they differ in warp/subgroup width, in how much shared memory pays off, in the ideal number of output elements a thread should carry (the unroll factor), in the block size that fills the machine. The reflex is to fork — keep a `matvec.amd` and a `matvec.nvidia`, each hand-tuned.

That reflex is the original sin the DSL exists to prevent, wearing a schedule’s clothes. Two forks are two kernels, maintained by two hands, drifting apart. The day someone reorders a loop in one fork “to help the unroll,” its accumulation order changes, its rounding changes, and now the same operation produces different bits on two backends — the precise failure a single-source DSL was supposed to make impossible ([5]). Worse, the drift is invisible until a model degenerates on one device and not another.

We want the performance a per-device schedule buys *without* the second kernel that buys the bug.

2 The idea: a schedule is a compile-time value

The DSL already gives one source lowered to every backend. The remaining fork — the schedule — can be folded into that one source if we treat the schedule not as *code* but as a *value*.

In `hanzo-kernel`, a kernel parameter marked `#[comptime]` is resolved at compile time, and the lowering engine *monomorphizes* the kernel on it: each distinct comptime value produces a distinct compiled kernel, specialized as if you had hand-written it. So we lift the schedule knobs to `#[comptime]` parameters of the one kernel (Listing 1). A “variant” is then not a forked file but a *choice of comptime tuple*. There is one source; only the chosen tuple differs per device.

Listing 1: One autotuned source. The unroll factor `rpt` (rows-per-thread) is a comptime knob; the block size is the host’s knob. Every `(rpt, block)` is a distinct compiled kernel yet bit-identical to the reference: the per-row fold is the same order, so only the thread→row mapping and occupancy differ.

```
#[kernel(targets(cuda, metal, vulkan, webgpu, cpu), unchecked)]
pub fn rms_norm_tuned<F: Float>(x: &Array<F>, w: &Array<F>, out: &mut Array<F>,
                                eps: &Array<F>, #[comptime] n: usize, #[comptime] rpt: usize) {
    let nrows = out.len() / n;
    let base = ABSOLUTE_POS * rpt;
    #[unroll]
    for r in 0..rpt {
        let row = base + r;
        if row < nrows {
            let b = row * n;
            let mut ss = F::new(0.0);
            for i in 0..n { let v = x[b + i]; ss += v * v; } // same fold, same order, every variant
            let denom = (ss / F::cast_from(n as u32) + eps[0]).sqrt();
            for i in 0..n { out[b + i] = x[b + i] / denom * w[i]; }
        }
    }
}
```

The critical invariant is stated in the comment and enforced by test: the knobs change the thread-to-work mapping and the occupancy, *not* the arithmetic. The per-row sum-of-squares is the same left fold in the same order for every `(rpt, block)`. Therefore all variants are **byte-identical** to each other and to the reference. The autotuner is choosing among numerically indistinguishable kernels; it cannot pick a “faster but slightly wrong” one, because there is no wrong one to pick.

3 The mechanism: three orthogonal values

Autotuning decomposes into three plain values, each testable in isolation (Listing 2).

- A **Variant** is a named schedule: a closure `f(iters) -> (output, ms)` that launches the schedule `iters` times after a fixed warmup and returns its output and mean milliseconds per dispatch. The same closure both *benchmarks* (many iters) and *executes* (one iter) — this is the crate’s established `*.bench` shape, so no separate “run” and “time” paths can disagree.
- A **Tuned** is the fluent builder: collect the variants for one `(op, shape-key)`, then `run`. It reads like the fusion builder — fluent, no macro, no ceremony.
- A **Tuner** is the cache: an in-memory `(device, op, key) -> winner` map backed by a per-device on-disk file. It is a *plain value* constructed over an explicit cache root, so it is fully unit-testable offline — no GPU, no environment, no global state. The process-wide default roots at the XDG cache directory.

Listing 2: The production surface. One source per op, autotuned per device, no forks. On a cache hit only the winner launches (once, to produce its output); the losers are never run.

```
let y = Tuned::new("rms_norm", format!("rows={rows},n={n}"))
    .variant("b64_r1", |it| rms_norm_tuned_bench(&c, x, w, rows, n, eps, 1, 64, it))
    .variant("b128_r1", |it| rms_norm_tuned_bench(&c, x, w, rows, n, eps, 1, 128, it))
    .variant("b256_r1", |it| rms_norm_tuned_bench(&c, x, w, rows, n, eps, 1, 256, it))
    .variant("b64_r2", |it| rms_norm_tuned_bench(&c, x, w, rows, n, eps, 2, 64, it))
    .run(&c); // times each once per (device,op,shape), caches the winner, thereafter runs it
```

The selection logic is deliberately small. On a cache *miss*, every variant is timed `TUNE_ITS` times (25), the fastest is kept, its winning name is recorded, and its already-computed output is returned — no redundant re-run. On a cache *hit*, only the winner is launched, once, to produce its output; the losers are never launched. If a cached winner name is no longer among the current variants (the set changed), that is treated as a miss and re-tuned rather than trusting a stale name.

The cache is a value, not a database. The on-disk format is one TSV file per device — `$XDGCACHE_HOME/hanzo-kernel/autotune/<device>.tsv` — each line `op\tkey\twinner`. No `serde`, no new dependency, no schema migration. Cross-process writes append and are idempotent (the loader keeps the last line for a key), which is exactly how an advisory autotune cache should behave: a hint that is safe to lose and safe to race on.

4 Evaluation: the variant tables and the offline proof

Two ops ship autotuned (Table 1). Both variant sets are proven on the CPU runtime, which needs no GPU to establish the two things under test: that the variants are numerically indistinguishable, and that the select/cache machinery works.

Op	Knobs	Variants (all byte-identical)
<code>rms_norm</code>	cube dim {64, 128, 256} × unroll {1, 2}	<code>b64_r1</code> <code>b128_r1</code> <code>b256_r1</code> <code>b64_r2</code>
<code>matvec_q8_dp4a</code>	cube dim {64, 128, 256} × vec. width {1, 2, 4, 8}	<code>b64_v1</code> <code>b64_v4</code> <code>b128_v2</code> <code>b256_v8</code>

Table 1: The autotune variant sets (`hanzo-kernel` 0.2.17). A knob value is a distinct compiled kernel; the tuner caches the winner per (`device`, `op`, `shape`). For `dp4a` `matvec` the vector-width knob is the ILP factor (groups per unrolled step); the per-group `int8` dot is accumulated in the same order for every width, so all four are byte-identical.

The committed tests assert, for both ops:

1. **Byte-identity.** Every schedule’s output has identical bits to the reference (RMSNorm: exactly equal; `dp4a`: byte-identical across variants and within `f32`-reorder tolerance 2×10^{-3} of the per-element oracle). The knobs do not touch the numerics.
2. **Select and cache.** A first call is a miss that benchmarks all four variants (`benched == 4`) and records the winner. A second call is a hit that benchmarks *zero* (`benched == 0`) and returns the winner’s output. A fresh `Tuner` over the same directory reloads the winner *from disk* and skips timing. A distinct shape-key is an independent decision. A shrunk variant set (winner absent) re-tunes rather than trusting a stale name.

Timing on the CPU proves the select/cache mechanism; a GPU run additionally makes the *winner* meaningful, since on the CPU the schedules are near-equivalent and the chosen winner is arbitrary-but-stable. The mechanism, not the specific winner, is what the offline tests pin down.

5 Why first-party, not the engine’s autotuner

The lowering engine (CubeCL) ships an autotuner; it was studied and declined for three concrete reasons, not out of not-invented-here. First, it is re-exported only from `cubeccl_runtime`, so adopting it would force a *second* engine crate into a facade architected so that only one line of one `Cargo.toml` names the engine at all — a deliberate “values, not places” boundary the whole DSL is built to preserve. Second, its persistent cache lives under `target/` (via an MD5 checksum and `serde`), not the XDG path this layer is required to use. Third, its generic-associated-type input machinery is a large surface for what is, in the end, “pick the fastest of N named closures.” The concepts above — variant, tuned set, cache keyed by device and shape — are *deliberately* the same as the engine’s; they are sized to this crate and pointed at XDG. A thin analog, not a new philosophy.

6 The takeaway

The schedule was the last place a backend forked a kernel, and the fork was the last hiding place for the divergence bug. Making the schedule a compile-time value folds that fork into the one source: the tuner enumerates numerically-identical variants, times them per device, and caches the winner, so the engine is autotuned and single-source at once. Speed is chosen; correctness is never spent. Combined with fusion as composition ([2]) and the oracle law ([5]), the schedule stops being a place where two implementations can disagree, because there is only ever one.

References

- [1] Hanzo AI. *One Source, Every Backend: The hanzo-kernel DSL*. Paper 07 (umbrella).
- [2] Hanzo AI. *Fusion Is Composition: Kernel Fusion as the Functor Law*. Paper 07.1.
- [3] Hanzo AI. *The Schedule Is a Value: Comptime-Specialized Autotuning*. Paper 07.2 (this paper).
- [4] Hanzo AI. *Intrinsic Islands: An Oracle-Anchored Escape Hatch*. Paper 07.3.
- [5] Hanzo AI. *One Oracle, Every Backend: The CPU Bit-Exactness Law*. Paper 07.4.
- [6] Hanzo AI. *When the Product Won’t Collapse: An int8 Tensor-Core MMQ Negative Result*. Paper 07.5.
- [7] Hanzo AI. *Verify, Then Delete: Collapsing a Kernel Cartesian Product*. Paper 07.6.